Classical Fourth-Order Runge–Kutta

Four-stage explicit integration

1 Butcher tableau and stages

The classical method has tableau

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

and, for $\dot{z} = f(t, z)$,

$$k_1 = f(t_n, z_n), \tag{1}$$
$$k_2 = f(t_n + h/2, z_n + hk_1/2), \tag{2}$$
$$k_3 = f(t_n + h/2, z_n + hk_2/2), \tag{3}$$
$$k_4 = f(t_n + h, z_n + hk_3), \tag{4}$$
$$z_{n+1} = z_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{5}$$

The non-autonomous stage times are part of the model.

2 Order and stability

The tableau satisfies the Runge–Kutta order conditions through order four; the local error is $\mathcal{O}(h^5)$ and the global error is $\mathcal{O}(h^4)$ for sufficiently smooth problems. On $y' = \lambda y$, its stability polynomial is

$$R(\xi) = 1 + \xi + \frac{\xi^2}{2} + \frac{\xi^3}{6} + \frac{\xi^4}{24}.$$

The finite stability region does not make RK4 an A-stable method.

3 Geometric properties

Classical RK4 is not symmetric and is not symplectic for a general Hamiltonian system. Its high local accuracy can nevertheless make short-time energy and area errors small. The distinction is structural: symplecticity is the exact identity

$$(D\Phi_h)^T \Omega D\Phi_h = \Omega,$$

not merely a small defect at one step size. The implementation exposes each complete step map to observers so local and accumulated defects can be measured.

4 Endpoint energy observation

The generic `EnergyObserver` evaluates physical H and analytic positive $\partial_t H$ at accepted endpoints and requested physical samples. It records energy differences without integrating momentum or using internal stages. A historical $H + \kappa$ study must explicitly request the separate stage-based diagnostic. Physical trajectories and solver equations are unchanged.

5 Implementation correspondence

The fixed tableau is declared in `src/methods/runge_kutta/rk4.py` and the inherited algorithm consumes the injected vector field $f(t, z)$. Main-grid steps are independent of the output schedule; shadow steps provide off-grid samples without changing the accepted trajectory or emitting observations. See the sibling runtime document for the exact result and diagnostic contract.

The recipe inherits stage calculations from `ExplicitRungeKutta`; `RungeKutta` supplies common step results and passive stage quadrature. The explicit algorithm consumes the immutable coefficients A, B and C without a separate implementation of this recipe.

6 Physical state and endpoint energy observations

Accepted states contain one physical vector $Z \in \mathbb{R}^{dN}$ and scalar time. GC vectors have $2N$ entries and FC vectors have $4N$; doubled workspaces are transient. InitialConditions supplies values and layout during preparation only. The numerical base retains functions, metadata, run status and copy count; execution owns the initial snapshot and output schedule. Supplied Jacobians are independent particle blocks (N, d, d); finite differences use dense matrices.

The generic `EnergyObserver` evaluates only

$$H_n = H(t_n, Z_n), \quad H_{n+1} = H(t_{n+1}, Z_{n+1}), \quad (\partial_t H)_n, \quad (\partial_t H)_{n+1}, \quad \Delta H_n = H_{n+1} - H_n.$$

It also evaluates these physical functions at requested saved samples, without consuming internal stages or integrating a momentum coordinate. The derivative is positive, analytic and partial at fixed state. GC differentiates its normalized Hamiltonian potential with `dt=1`; spatial splines and Fourier time factors retain their original scaling. This is not the secant $\Delta H_n/h$. Variation of a time-dependent Hamiltonian is not a numerical conservation error.

Specialized historical generalized-energy studies separately reconstruct $\dot{\kappa} = -\partial_t H$ from retained numerical stages. Their signed weights, physical normalization and archived diagnostics keep their original meaning. They are not part of generic energy observation. Host and JAX execution share the fixed numerical map and physical-only accepted state. JAX evaluates endpoint energy on the device without a momentum accumulator.

7 CPU and JAX execution of the same map

The execution configuration selects either the default NumPy/SciPy CPU driver or a JAX driver on one CPU or GPU device. Both consume the same RK4 stage and passive-quadrature formulas. The potential uses the same fitted spline knots and coefficients in both implementations. JAX requires explicit double precision; different floating-point operation ordering can produce round-off differences, so equivalence does not mean bitwise identity.

For independent particles, stage operations are batched over the particle index. The four stages retain their dependencies, and successive time steps remain sequential inside a compiled loop. The accepted state and requested sample buffer remain on the chosen device until the completed result is transferred to the standard immutable NumPy solution. Independent shortened maps still provide off-grid samples without altering accepted main states. The passive momentum and sampled Hamiltonian are evaluated on that device.

The compiled route currently covers the built-in guiding-centre and full-cyclotron equations. Custom dynamics, method subclasses and Python step callbacks or progress reporting require the CPU route. Device choice does not change the order, stability region or geometric properties derived above. First-call compilation must be distinguished from warmed integration timings, which include transfers and output construction in the test study.